1. Introduction

The ocean hides an incredible world that most of us never get to see directly. Through underwater photography, people try to explore and capture this world — whether it's for scientific research, environmental studies, or simply to show the natural beauty that lies beneath the surface. These photographs often play an important role in helping scientists study marine life, track ecological changes, and document the health of underwater ecosystems.

Still, taking good pictures underwater is far from simple. The deeper you go, the less light there is, and the water starts to absorb certain colors — especially reds and yellows — which makes everything look bluish or greenish. On top of that, small particles floating in the water scatter light, making the images look foggy or unclear. Because of this, most underwater photos lose their sharpness and natural colors, making it difficult to see the real beauty of marine life or use the images accurately for research.

In the last few years, **artificial intelligence (AI)** and **deep learning** have started to change how we deal with these problems. Modern algorithms, such as **Convolutional Neural Networks (CNNs)** and **Generative Adversarial Networks (GANs)**, can learn from thousands of images to automatically fix distortion and color loss. Instead of adjusting every image manually, these models can figure out how to balance colors, clear haze, and bring back lost details — almost like how our eyes naturally adjust to different lighting.

That's what inspired **AquaVision AI**, a project designed to make underwater images look more natural and lifelike. The main idea is to use deep learning to enhance underwater photographs by correcting their colors, improving clarity, and removing haze. This not only makes the pictures more visually appealing but also helps divers, researchers, and photographers get images that are closer to what they actually see underwater. AquaVision AI hopes to bridge the gap between technology and the natural beauty of the ocean — showing the underwater world the way it truly is.

1.1 Objective of the Project

The primary objective of **AquaVision AI** is to design and develop an intelligent deep learning–based system that can **automatically enhance underwater images** by correcting color distortions, reducing haze, and improving overall image clarity.

The specific goals include:

1. **Developing a deep learning model** (CNN or GAN) capable of learning the mapping between degraded and high-quality underwater images.

2. **Implementing preprocessing techniques** such as color correction, dehazing, and noise reduction to improve model input quality.
3. **Utilizing benchmark underwater image datasets** like UIEB and EUVP for model training and validation.
4. **Evaluating enhanced images** using quantitative metrics such as PSNR, SSIM, and UIQM to ensure objective quality improvement.
5. **Creating a user-friendly interface** that allows users to upload underwater images and instantly obtain enhanced versions.

Through these objectives, AquaVision AI aims to bridge the gap between artistic underwater photography and scientifically accurate marine imaging.

1.2 Description of the Project

AquaVision AI is a deep learning–based system designed to automatically improve the quality of underwater photographs. The project focuses on solving common issues such as poor visibility, faded colors, and low contrast that usually occur when capturing images underwater. Instead of relying on manual editing or traditional filters, AquaVision AI uses artificial intelligence to study patterns in degraded underwater images and learn how to restore them closer to their natural appearance.

The system is built around **Convolutional Neural Networks (CNNs)** or **Generative Adversarial Networks (GANs)** — advanced AI models that can understand complex visual distortions. These models are trained using popular underwater image datasets such as **UIEB (Underwater Image Enhancement Benchmark)** and **EUVP (Enhancement of Underwater Visual Perception)**. By analyzing pairs of raw and enhanced images, the model gradually learns how to correct color loss, remove haze, and sharpen fine details.

The final output is an AI tool that can take any underwater photograph and enhance it instantly. Along with this, the project includes a simple **web or mobile interface** where users can upload their images and receive enhanced results in real time. The system is designed to be efficient, easy to use, and capable of producing results that look natural while maintaining scientific accuracy.

1.3 Scope of the Project

The scope of **AquaVision AI** extends beyond just improving the look of underwater images. It aims to create a bridge between **artistic photography, marine research, and technology**. The system can be used by **divers, marine biologists, photographers, filmmakers, and researchers** who need clearer and more accurate visual data from underwater environments.

In its current form, the project focuses on still images, but the same approach can later be expanded to **underwater videos** and **real-time enhancement systems** used in remotely operated vehicles (ROVs) or underwater drones. This would help in exploring deep-sea regions where lighting conditions are poor and manual editing is not feasible.

The project also supports **scientific documentation**, helping experts analyze coral health, aquatic habitats, or underwater pollution with improved visibility and color accuracy. In short, the scope of AquaVision AI is not limited to image enhancement alone — it represents a step forward toward **intelligent underwater vision systems** that combine deep learning, marine imaging, and environmental sustainability.

2. System Description

AquaVision AI is designed as an intelligent image enhancement system that uses **deep learning** to restore and improve underwater photographs automatically. The system's primary goal is to correct the color imbalance, haze, and blurriness that occur due to light scattering and absorption in underwater environments.

At its core, AquaVision AI consists of three major components:

1. **Preprocessing Module:** Handles initial image adjustments such as white balance correction, noise removal, and normalization to prepare the data for the model.
2. **Deep Learning Model:** A **CNN or GAN-based network** that processes underwater images and generates enhanced outputs by learning the mapping between degraded and high-quality images.
3. **User Interface:** A simple **web or mobile platform** that allows users to upload their images and instantly receive improved results. The interface is designed to be user-friendly, fast, and accessible even to non-technical users.

The system follows a streamlined workflow. First, the input image is passed through the preprocessing stage to clean and normalize it. Then, the deep learning model enhances the

image by restoring lost colors, increasing contrast, and removing haze. Finally, the processed image is displayed or downloaded through the interface.

AquaVision AI aims to make **underwater image restoration** quick, automated, and efficient — providing high-quality visual output with minimal user intervention.

2.1 Customer/User Profiles

The AquaVision AI system is designed to serve a variety of users who work with or capture underwater visuals. These include:

1. **Marine Researchers and Scientists:**
   They require clear and accurate images for studying marine biodiversity, coral reefs, and underwater ecosystems. Enhanced images help in analyzing textures, species identification, and monitoring environmental changes over time.
2. **Professional and Amateur Photographers:**
   Photographers who capture underwater scenes often struggle with poor lighting and color loss. AquaVision AI provides them with a tool to automatically enhance their photos, saving editing time and improving overall image quality.
3. **Scuba Divers and Ocean Explorers:**
   Recreational divers who use underwater cameras can use AquaVision AI to enhance their personal footage, making their underwater experiences look more vibrant and lifelike.
4. **Environmental Organizations:**
   NGOs and conservationists working on underwater projects can use enhanced visuals for reports, campaigns, and awareness programs where image clarity and realism are essential.
5. **Filmmakers and Documentary Creators:**
   Underwater footage used in documentaries or films often needs post-processing. AquaVision AI can significantly reduce manual color grading work, producing clean and realistic visuals automatically.

Overall, AquaVision AI serves anyone who wishes to **restore the natural beauty and clarity of underwater images**, whether for research, art, or personal exploration.

2.2 Assumptions and Dependencies

While developing **AquaVision AI**, several assumptions and dependencies are considered to ensure the system performs as expected. These help define the boundaries and technical environment in which the project operates.

Assumptions:

1. The input images are captured using standard underwater cameras or devices capable of producing digital images with basic clarity and resolution.
2. The images provided for enhancement contain visible details, even if they are distorted or hazy, so the AI model can learn and restore them effectively.
3. Users will have access to a stable internet connection when using the web-based or mobile interface for uploading and processing images.
4. The datasets (such as **UIEB** and **EUVP**) used for model training are representative enough of different underwater environments — shallow, deep, clear, and turbid waters.
5. The computational resources, such as **GPU-enabled systems**, are available during training to handle high-resolution image processing efficiently.

Dependencies:

1. The system depends on **deep learning frameworks** like TensorFlow or PyTorch for training and model implementation.
2. **OpenCV** and other image-processing libraries are required for preprocessing tasks such as noise reduction and color normalization.
3. The performance of the model depends on the **quality and diversity of datasets** used during training.
4. The web or mobile platform will rely on **cloud services or local hosting** for storing and processing images.
5. Continuous evaluation and updates will depend on the availability of new underwater datasets to improve the model's generalization.

2.3 Functional Requirements

The functional requirements describe what the **AquaVision AI** system is expected to do and how users will interact with it. These define the key operations, processes, and features necessary for the system to function effectively.

1. Image Input and Upload

- The system should allow users to upload underwater images through a web or mobile interface.
- Supported formats include **JPEG, PNG, and TIFF**.
- Users can upload single or multiple images for batch processing.

2. Preprocessing

- The system should automatically adjust brightness, contrast, and white balance before enhancement.
- It should remove noise and normalize image dimensions for consistent input to the model.

3. Image Enhancement using Deep Learning

- The core model (CNN or GAN) should process the image and generate an enhanced version with improved visibility, restored colors, and reduced haze.
- The model should automatically detect and correct overexposed or underexposed regions.

4. Quality Evaluation

- The system should calculate and display quality metrics such as **PSNR, SSIM, and UIQM** for each enhanced image.
- These metrics help verify that the enhanced image has improved over the original in terms of clarity and visual quality.

5. Output and Download

- The enhanced image should be displayed in the interface for visual comparison (before vs. after).
- Users should have the option to download the enhanced image in the same or a different file format.

6. User Interface Requirements

- The interface should be intuitive and responsive, allowing users to interact easily with minimal steps.

- It should display progress while the image is being processed and notify users once enhancement is complete.

7. Model Training and Updating

- The system should support retraining or fine-tuning the model with new datasets to improve performance over time.
- Admins or developers should be able to update model weights and retrain through the backend.

2.4 Non-Functional Requirements

While functional requirements describe what **AquaVision AI** should do, non-functional requirements define **how well** the system performs its functions. These include aspects like performance, reliability, usability, scalability, and maintainability, which ensure that the system is efficient, stable, and user-friendly.

1. Performance Requirements

- The system should be able to process and enhance an underwater image within a few seconds, depending on its resolution.
- The AI model must deliver consistent performance across various types of underwater conditions such as clear, murky, or deep-water images.
- GPU acceleration or optimized processing techniques should be used to handle large datasets and high-resolution images efficiently.

2. Reliability and Accuracy

- The system should consistently produce enhanced images without major visual artifacts or distortions.
- Model predictions must be reliable, with noticeable improvement in color balance, sharpness, and clarity in at least **90%** of test cases.
- In case of processing errors or corrupted uploads, the system should handle them gracefully by providing meaningful error messages.

3. Usability Requirements

- The user interface should be intuitive and require minimal technical knowledge.

- Clear visual indicators (such as progress bars or preview thumbnails) should guide users during image upload and enhancement.
- The system should support both light and dark themes for better accessibility.
- Documentation or short tutorials should be available to help first-time users understand how to use the system effectively.

4. Scalability

- The architecture should be designed to handle an increasing number of users and images without significant delays.
- The system should support deployment on cloud platforms (e.g., AWS, Azure, or Google Cloud) to enable large-scale processing and storage.
- Future extensions, such as video enhancement or integration with underwater drones, should be feasible without major structural changes.

5. Security and Privacy

- User-uploaded images must be handled securely and deleted automatically after processing to ensure data privacy.
- Secure communication protocols (e.g., HTTPS) should be used for all data transfers.
- Only authorized users or admins should have access to the model training and dataset management modules.

6. Maintainability and Upgradability

- The codebase should be modular and well-documented to simplify debugging and updates.
- The system should allow developers to easily retrain or update the model when new underwater datasets become available.
- Regular maintenance checks should ensure compatibility with newer library versions and hardware configurations.

7. Portability

- The system should be designed to run on multiple platforms, including web browsers and mobile devices.
- Cross-platform compatibility ensures that users can access the application from different operating systems without performance issues.

3. Design

The design of **AquaVision AI** focuses on creating a system that is accurate, efficient, and user-friendly. The system is built around deep learning techniques that enhance underwater images while keeping the workflow simple for users. The design stage involves defining how each component interacts, how data flows through the system, and how performance is optimized at every step.

The overall design ensures that the system not only restores the visual quality of underwater images but also maintains flexibility for future extensions—such as real-time enhancement, integration with underwater drones, or video-based processing. It balances computational performance with accessibility so that both technical and non-technical users can benefit from the tool.

3.1 System Design

The **AquaVision AI** system follows a modular architecture that connects several components working together to achieve image enhancement. Each module has a specific role, ensuring that the overall process remains efficient and scalable. The major components are:

1. Input and Data Collection Layer

This layer handles the **input of underwater images**. Users upload images through the web or mobile interface. These images are then validated for file type, resolution, and quality before being processed. The system supports multiple formats such as **JPEG, PNG, and TIFF**.

Additionally, during model training, large-scale datasets such as **UIEB (Underwater Image Enhancement Benchmark)** and **EUVP (Enhancement of Underwater Visual Perception)** are used. These datasets provide pairs of low-quality and high-quality underwater images for supervised learning.

---

2. Preprocessing Module

Before feeding the image into the deep learning model, it goes through several **preprocessing steps** to improve the input quality:

- **Noise Reduction:** Removes small particles or image artifacts using filters.
- **Color Normalization:** Adjusts the white balance and intensity distribution.
- **Contrast Enhancement:** Improves visibility in darker regions of the image.

- **Resizing:** Ensures consistent image dimensions for model compatibility.

These preprocessing steps help the neural network perform better during enhancement by providing cleaner and more balanced input data.

---

3. Deep Learning Model (Core Processing Unit)

This is the heart of AquaVision AI. The system uses a **Convolutional Neural Network (CNN)** or a **Generative Adversarial Network (GAN)** to automatically enhance underwater images.

- The **CNN model** focuses on restoring lost features such as edges, textures, and colors.
- The **GAN model** goes a step further by generating realistic enhancements through an adversarial learning process, where a generator and discriminator work together to produce more natural-looking outputs.

The model learns to map degraded underwater images to their enhanced versions by analyzing thousands of image pairs during training. Once trained, it can process new images instantly.

---

4. Post-Processing Module

After enhancement, the output image passes through a post-processing stage to refine the final result. This step may include:

- Fine-tuning brightness and contrast.
- Removing any remaining noise or halos.
- Sharpening image details for better clarity.

This ensures that the final image looks natural and visually appealing, maintaining both aesthetic and scientific accuracy.

---

5. User Interface (Frontend Layer)

The user interface provides a simple and interactive experience. Users can upload images, view enhancement progress, and compare the **before-and-after** results. The interface is designed to be:

- **Easy to navigate** (minimal steps to process an image).
- **Responsive** (works on both web and mobile devices).
- **Visually clean**, displaying side-by-side image comparisons and downloadable results.

Technologies such as **HTML, CSS, JavaScript**, and frameworks like **Flask**, **Streamlit**, or **React.js** can be used to build the interface.

---

6. Evaluation and Metrics Module

The enhanced images are evaluated using standard image quality metrics, including:

- **PSNR (Peak Signal-to-Noise Ratio)** – measures the clarity of the image.
- **SSIM (Structural Similarity Index)** – measures structural similarity between original and enhanced images.
- **UIQM (Underwater Image Quality Measure)** – assesses overall underwater image quality (color, sharpness, contrast).

These metrics help ensure that AquaVision AI consistently produces scientifically valid and visually pleasing results.

---

7. Output Layer

Once processing is complete, the enhanced image is displayed on the interface with an option to **download** it in the desired format. The system also provides a comparison view showing how much improvement has been achieved compared to the original image.

System Workflow Summary

1. User uploads an underwater image.
2. Image passes through the preprocessing stage.
3. Deep learning model enhances the image.
4. Post-processing refines the output.
5. Results are displayed and evaluated using metrics.
6. User downloads or saves the enhanced image.

3.1.1 E–R Diagram

This section models the core data entities and their relationships for **AquaVision AI**—from user uploads to model-driven enhancement and quality evaluation. The design separates

operational data (user images and jobs) from **training data** (datasets and model versions), making the system maintainable and auditable.

Entities (with key attributes)

1. **User**
    - *user_id (PK)*, name, email, role {researcher/photographer/admin}, organization, created_at, status
2. **Image**
    - *image_id (PK)*, *user_id (FK→User)*, filename, mime_type, width, height, capture_device, gps_location, uploaded_at, checksum
3. **PreprocessConfig**
    - *config_id (PK)*, white_balance_mode, denoise_method, resize_long_edge, contrast_boost, notes, created_at
4. **EnhancementJob**
    - *job_id (PK)*, *image_id (FK→Image)*, *user_id (FK→User)*, *config_id (FK→PreprocessConfig, nullable)*, *model_id (FK→ModelVersion)*
    - submitted_at, started_at, finished_at, status {queued/running/succeeded/failed}, processing_ms, error_message
5. **EnhancementResult**
    - *result_id (PK)*, *job_id (FK→EnhancementJob, unique)*, output_filename, output_width, output_height, storage_uri, checksum, preview_uri
6. **QualityMetric**
    - *metric_id (PK)*, *result_id (FK→EnhancementResult)*, psnr, ssim, uiqm, uciqe, computed_at
7. **ModelVersion**
    - *model_id (PK)*, model_name, model_type {CNN/GAN}, version_tag, framework {PyTorch/TensorFlow}, trained_on, notes, artifact_uri
8. **Dataset**
    - *dataset_id (PK)*, name {UIEB/EUVP/…}, source, split_summary, num_images, license, ingestion_date
9. **ModelDataset** *(junction for many-to-many)*

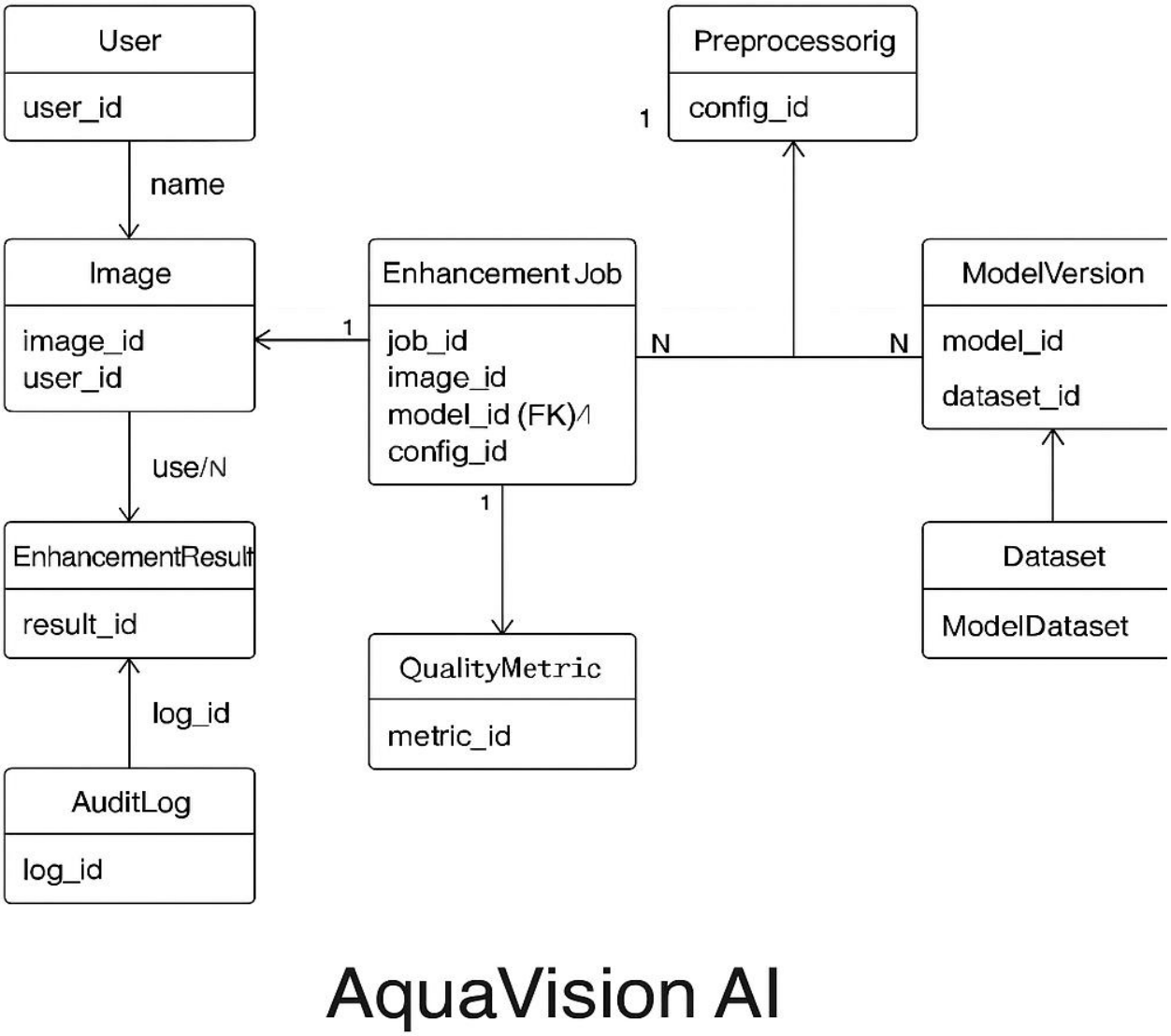
- *model_id (FK→ModelVersion)*, *dataset_id (FK→Dataset)*, usage {train/val/test}, notes
- *(composite PK: model_id + dataset_id + usage)*

10. **AuditLog** *(optional but recommended)*
    - *log_id (PK)*, *user_id (FK→User)*, action, target_type, target_id, detail, created_at, ip

---

Relationships (cardinalities)

- **User 1 — N Image**: one user can upload many images.
- **User 1 — N EnhancementJob**: jobs are requested by a user.
- **Image 1 — N EnhancementJob**: an image may be enhanced multiple times (different configs/models).
- **PreprocessConfig 1 — N EnhancementJob**: a job may reference one config; multiple jobs can reuse a config. *(Optional for jobs)*
- **ModelVersion 1 — N EnhancementJob**: each job uses exactly one model version.
- **EnhancementJob 1 — 1 EnhancementResult**: each job yields at most one result.
- **EnhancementResult 1 — N QualityMetric**: multiple metrics can be computed per result (PSNR, SSIM, UIQM, etc.).
- **ModelVersion N — M Dataset**: models may be trained/validated on multiple datasets and vice-versa (via **ModelDataset**).
- **User 1 — N AuditLog**: actions are recorded per user.



3.1.2 DFDs

A. Level-0 DFD (Context Diagram)

Purpose: Shows AquaVision AI as a single process and how it interacts with external entities.

External Entities

- **E1 User** (diver/photographer/researcher)
- **E2 Admin** (model/ops management)

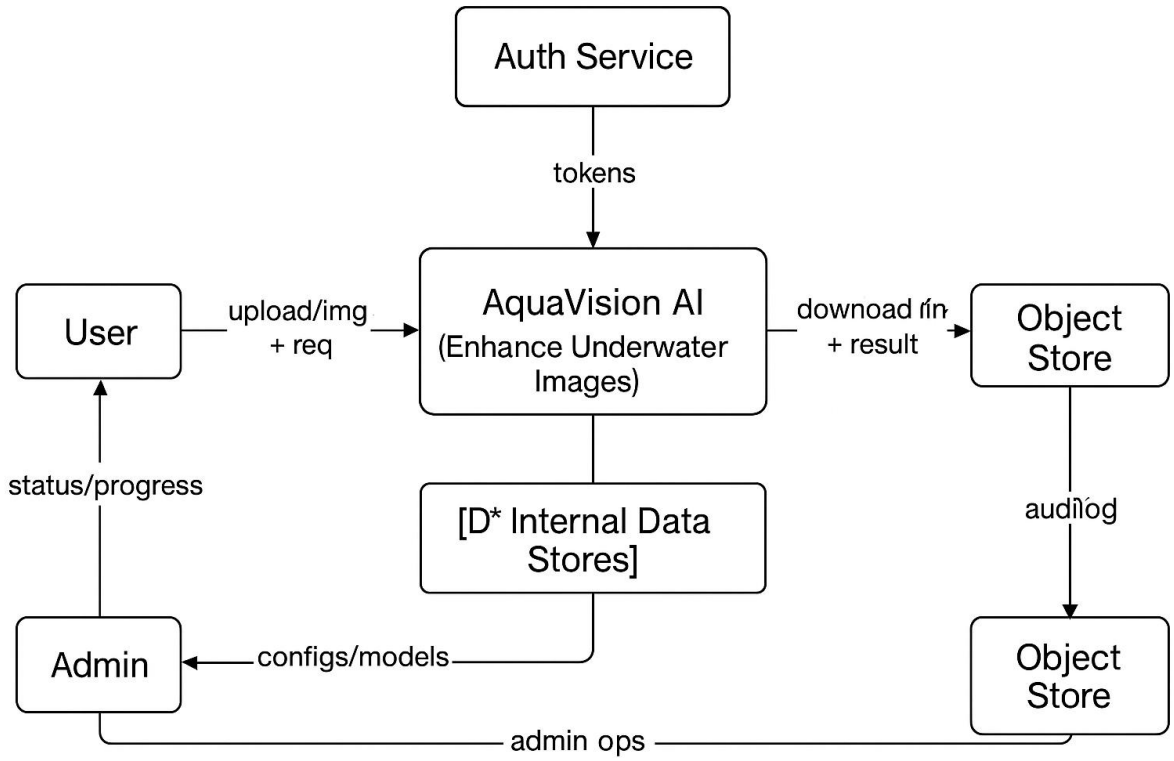
- **E3 Auth Service** (login/session)
- **E4 Object Storage/CDN** (stores originals & results; returns download links)

Process

- **P0 AquaVision AI System**

Data Stores (abstracted at L0)

- *D Internal Data Stores** (images, jobs, models, metrics) — shown as a single block at context level.



B. Level-1 DFD (Decomposition of P0)

Breaks the system into major processes and concrete data stores.

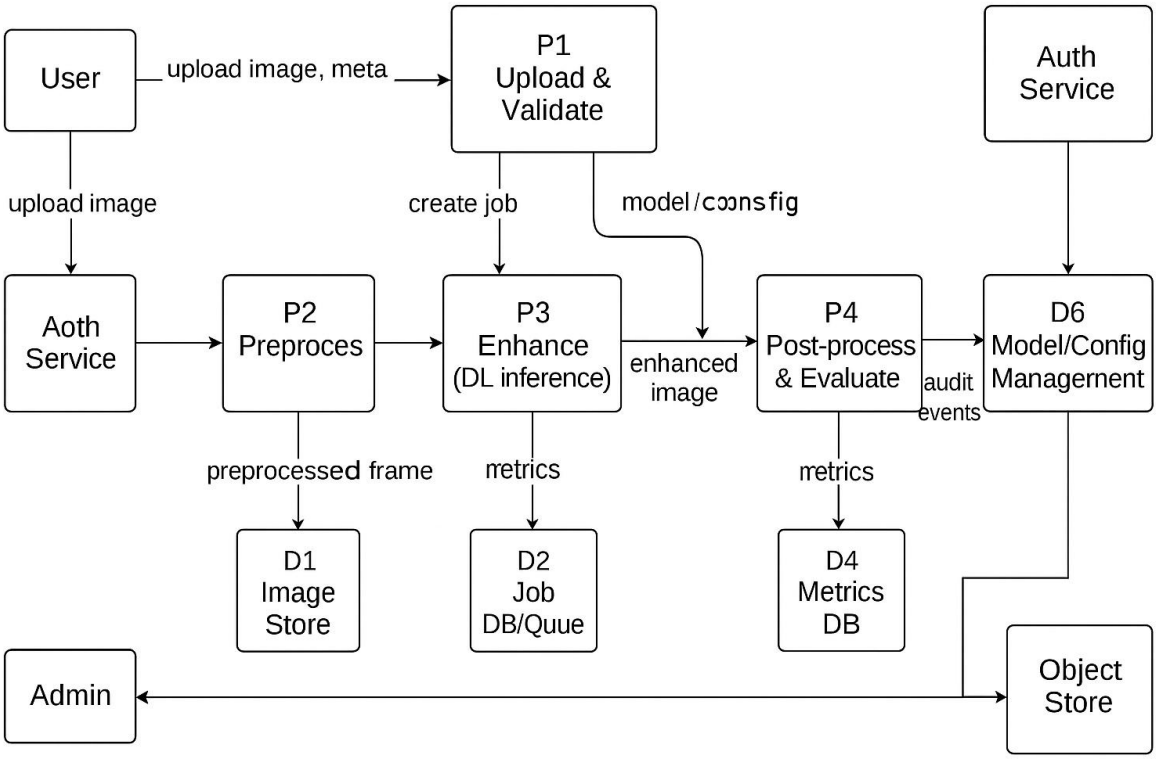
Processes

- **P1 Upload & Validate**
- **P2 Preprocess**
- **P3 Enhance (DL Inference)**

- **P4 Post-process & Evaluate**
- **P5 Deliver & Notify**
- **P6 Model/Config Management** (admin)

Data Stores

- **D1 Image Store** (originals)
- **D2 Job Queue / Job DB**
- **D3 Result Store** (enhanced images)
- **D4 Metrics DB** (PSNR/SSIM/UIQM/UCIQE)
- **D5 Model Registry** (versions, configs)
- **D6 Audit Log**



C. Level-2 DFD (Drill-down of P3: Enhance / DL Inference)

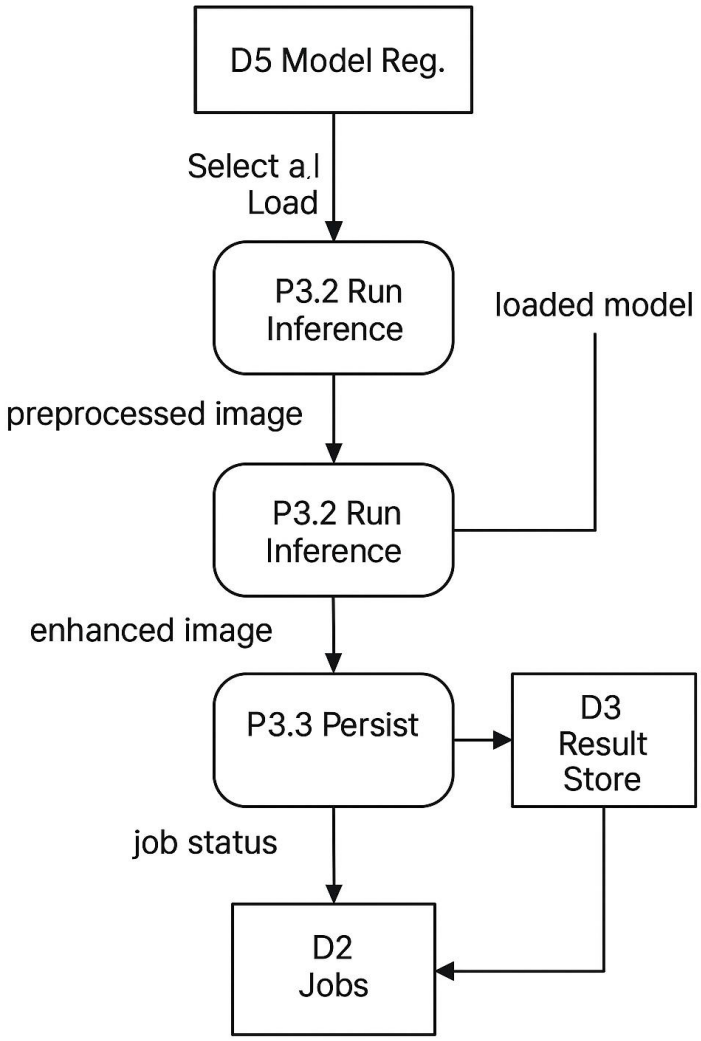
Details the internal steps of the core enhancement process.

Sub-Processes

- **P3.1 Select Model & Load Weights**
- **P3.2 Run Inference (CNN/GAN)**
- **P3.3 Persist Output**

Stores/Entities

- **D5 Model Registry** (archived weights/configs)
- **D3 Result Store**
- **D2 Jobs** (state updates)



D. Data Dictionary (flows at a glance)

- **Upload Image:** {file, filename, mime_type, width, height, capture_device, gps?, user_id}
- **Job Ticket:** {job_id, image_id, user_id, model_id?, config_id?, status, timestamps}
- **Preprocessed Frame:** {tensor/array, normalization_params, resize_info}
- **Model Spec:** {model_id, model_type, version_tag, weights_uri, preprocessing_requirements}
- **Enhanced Output:** {result_id, job_id, output_uri, dims, checksum}
- **Quality Metrics:** {psnr, ssim, uiqm, uciqe, computed_at}
- **Delivery Payload:** {job_id, status, preview_uri, download_link, metrics_summary}

3.2 Database Design

The database for **AquaVision AI** separates **operational runtime data** (uploads, jobs, results, metrics) from **ML governance data** (models, datasets, audit). The design follows **3NF** to reduce redundancy, preserve integrity, and support reproducible experiments.

3.2.1 Design Goals

- **Integrity & traceability:** Every enhanced image is reproducibly linked to its source image, preprocessing config, and model version.
- **Performance:** Fast lookups by user, job status, and time; efficient storage of large binaries via object storage (URIs only).
- **Security & privacy:** User data isolated; least-privilege access; minimal retention of uploads as configured.
- **Scalability:** Partitionable “hot” tables (jobs, metrics) by date; append-only audit.

3.2.2 Core Tables (normalized)

Types below assume PostgreSQL; adapt easily to MySQL/SQL Server.

1) users

- user_id BIGSERIAL PK
- name TEXT NOT NULL

- email CITEXT UNIQUE NOT NULL
- role TEXT CHECK (role IN ('researcher','photographer','admin'))
- organization TEXT
- created_at TIMESTAMPTZ DEFAULT now()
- Index: (email), (role)

2) images

- image_id BIGSERIAL PK
- user_id BIGINT FK → users(user_id) ON DELETE CASCADE
- filename TEXT, mime_type TEXT, width INT, height INT
- capture_device TEXT, gps_point GEOGRAPHY(Point) *(optional)*
- uploaded_at TIMESTAMPTZ DEFAULT now()
- checksum TEXT UNIQUE *(dedupe)*
- storage_uri TEXT NOT NULL *(object storage path)*
- Index: (user_id, uploaded_at DESC), (checksum) UNIQUE

3) preprocess_configs

- config_id BIGSERIAL PK
- white_balance_mode TEXT
- denoise_method TEXT
- resize_long_edge INT, contrast_boost NUMERIC(4,2)
- notes TEXT, created_at TIMESTAMPTZ DEFAULT now()
- Optional **natural key**: hash of parameters for dedupe.

4) model_versions

- model_id BIGSERIAL PK
- model_name TEXT NOT NULL
- model_type TEXT CHECK (model_type IN ('CNN','GAN'))
- version_tag TEXT NOT NULL
- framework TEXT CHECK (framework IN ('PyTorch','TensorFlow'))
- trained_on TIMESTAMPTZ

- artifact_uri TEXT NOT NULL *(weights bundle)*
- notes TEXT
- Constraint: UNIQUE (model_name, version_tag)

5) datasets

- dataset_id BIGSERIAL PK
- name TEXT NOT NULL *(e.g., UIEB, EUVP)*
- source TEXT, license TEXT
- num_images INT, ingestion_date DATE
- split_summary JSONB

6) model_datasets *(N:M junction)*

- model_id BIGINT FK → model_versions
- dataset_id BIGINT FK → datasets
- usage TEXT CHECK (usage IN ('train','val','test'))
- notes TEXT
- **PK:** (model_id, dataset_id, usage)

7) enhancement_jobs

- job_id BIGSERIAL PK
- image_id BIGINT FK → images ON DELETE CASCADE
- user_id BIGINT FK → users ON DELETE SET NULL
- config_id BIGINT FK → preprocess_configs
- model_id BIGINT FK → model_versions
- submitted_at TIMESTAMPTZ DEFAULT now()
- started_at TIMESTAMPTZ, finished_at TIMESTAMPTZ
- status TEXT CHECK (status IN ('queued','running','succeeded','failed'))
- processing_ms INT, error_message TEXT
- Indexes:
    - (status, submitted_at) for queue workers
    - (image_id) for job history per image

    - (user_id, submitted_at DESC) for user dashboards

8) enhancement_results

- result_id BIGSERIAL PK
- job_id BIGINT UNIQUE FK → enhancement_jobs ON DELETE CASCADE
- output_width INT, output_height INT
- storage_uri TEXT NOT NULL, checksum TEXT
- preview_uri TEXT
- Index: (checksum), (job_id UNIQUE)

9) quality_metrics

- metric_id BIGSERIAL PK
- result_id BIGINT FK → enhancement_results ON DELETE CASCADE
- psnr NUMERIC(6,3), ssim NUMERIC(5,4)
- uiqm NUMERIC(6,3), uciqe NUMERIC(6,3)
- computed_at TIMESTAMPTZ DEFAULT now()
- Index: (result_id), (computed_at DESC)

10) audit_logs

- log_id BIGSERIAL PK
- user_id BIGINT FK → users ON DELETE SET NULL
- action TEXT *('LOGIN', 'CREATE_JOB', 'RETRAIN_MODEL', ...)*
- target_type TEXT, target_id BIGINT
- detail JSONB, created_at TIMESTAMPTZ DEFAULT now()
- Index: (user_id, created_at DESC), (target_type, target_id)

3.2.3 Keys, Constraints & Integrity

- **Foreign keys** enforce referential integrity across jobs, results, and metrics.
- **Unique constraints** on (model_name, version_tag) and checksum enable deterministic model selection and duplicate detection.
- **Check constraints** keep enumerations valid (status, role, usage, model_type).
- **ON DELETE** rules:

    - Deleting an image cascades to its jobs/results/metrics.
    - Deleting a user preserves jobs via SET NULL for auditability.

3.2.4 Indexing & Query Patterns

- Dashboards: jobs by (user_id, submitted_at DESC); images by (user_id, uploaded_at).
- Worker queue: partial index WHERE status IN ('queued','running').
- Analytics: metrics by computed_at for time-series charts; materialized view joining jobs + results + metrics for reports.
- Search: trigram index on model_name/version_tag (Postgres pg_trgm) if you support fuzzy lookup.

3.2.5 Storage Strategy

- **Large binaries (images, weights)** are **not** stored in the DB; only URIs and checksums are kept. Object storage (S3, GCS, MinIO) handles files, with lifecycle rules for archival/deletion.
- **JSONB** used sparingly for flexible fields (e.g., split_summary, detail).

3.2.6 Partitioning & Retention (optional)

- **Range partition** enhancement_jobs and quality_metrics by month on submitted_at/computed_at for high volume.
- **Retention policy examples:**
    - Originals: 90 days (configurable).
    - Results: 180 days or until explicitly pinned by the user.
    - Audit logs: 365+ days.

3.2.7 Security & Privacy

- Row-level access: users can only read their images, jobs, results, metrics. Admins have broader scopes.
- Encrypt at rest (DB + object store) and in transit (TLS).
- Rotate signed URLs for downloads; avoid public buckets.
- PII minimalism: only store essential user fields.

3.2.8 Sample DDL Snippet (PostgreSQL)

```
CREATE TABLE users (
  user_id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  email CITEXT UNIQUE NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('researcher','photographer','admin')),
  organization TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE images (
  image_id BIGSERIAL PRIMARY KEY,
  user_id BIGINT REFERENCES users(user_id) ON DELETE CASCADE,
  filename TEXT, mime_type TEXT, width INT, height INT,
  capture_device TEXT, storage_uri TEXT NOT NULL,
  checksum TEXT UNIQUE, uploaded_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE model_versions (
  model_id BIGSERIAL PRIMARY KEY,
  model_name TEXT NOT NULL,
  model_type TEXT CHECK (model_type IN ('CNN','GAN')),
  version_tag TEXT NOT NULL,
  framework TEXT CHECK (framework IN ('PyTorch','TensorFlow')),
  trained_on TIMESTAMPTZ, artifact_uri TEXT NOT NULL,
  notes TEXT,
  UNIQUE (model_name, version_tag)
);
```

```
CREATE TABLE enhancement_jobs (
  job_id BIGSERIAL PRIMARY KEY,
  image_id BIGINT REFERENCES images(image_id) ON DELETE CASCADE,
  user_id BIGINT REFERENCES users(user_id) ON DELETE SET NULL,
  config_id BIGINT,
  model_id BIGINT REFERENCES model_versions(model_id),
  submitted_at TIMESTAMPTZ DEFAULT now(),
  started_at TIMESTAMPTZ, finished_at TIMESTAMPTZ,
  status TEXT CHECK (status IN ('queued','running','succeeded','failed')),
  processing_ms INT, error_message TEXT
);

CREATE TABLE enhancement_results (
  result_id BIGSERIAL PRIMARY KEY,
  job_id BIGINT UNIQUE REFERENCES enhancement_jobs(job_id) ON DELETE CASCADE,
  output_width INT, output_height INT,
  storage_uri TEXT NOT NULL, checksum TEXT, preview_uri TEXT
);

CREATE TABLE quality_metrics (
  metric_id BIGSERIAL PRIMARY KEY,
  result_id BIGINT REFERENCES enhancement_results(result_id) ON DELETE CASCADE,
  psnr NUMERIC(6,3), ssim NUMERIC(5,4),
  uiqm NUMERIC(6,3), uciqe NUMERIC(6,3),
  computed_at TIMESTAMPTZ DEFAULT now()
);
```

3.2.9 Migration & Backups

- Use a migration tool (Flyway/Prisma/Knex) for versioned schema changes.
- Nightly full backups + WAL archiving (Postgres) with restore drills.
- Keep **schema version** in a small table (schema_version) for CI/CD gating.

4. Scheduling and Estimates

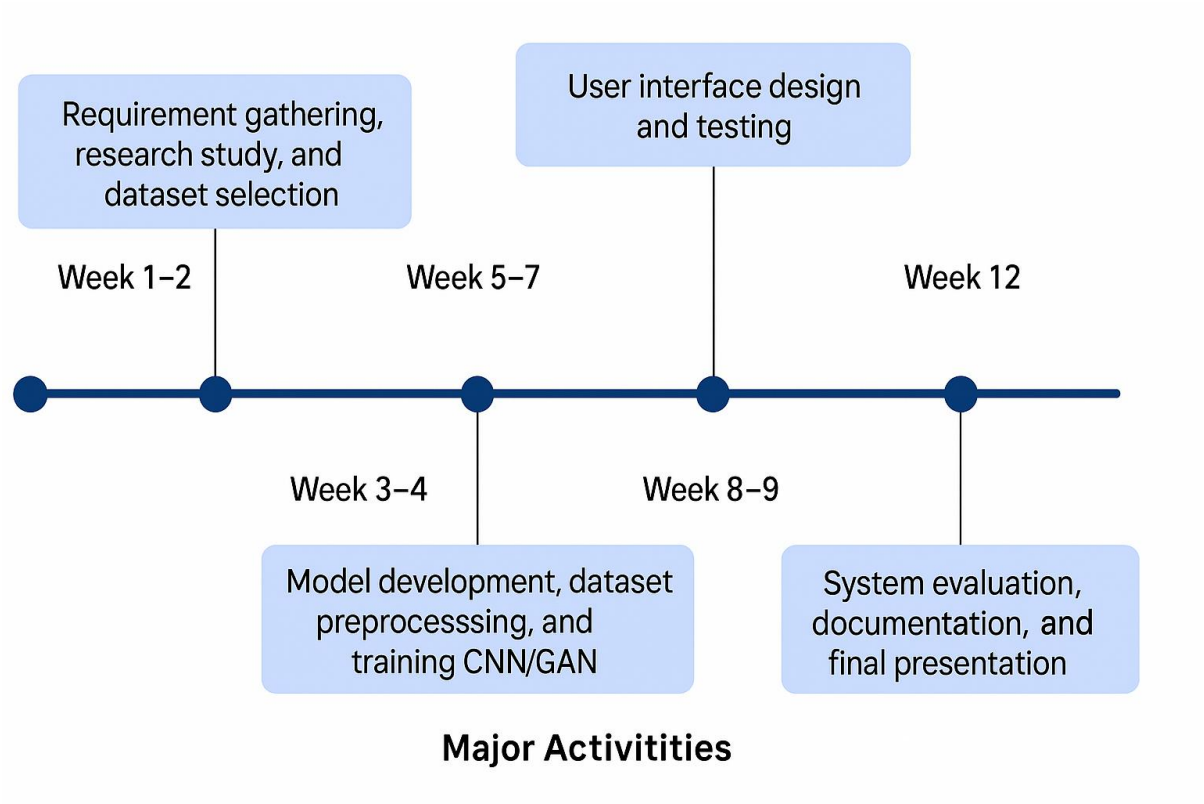
The success of **AquaVision AI** depends on structured planning, systematic development, and timely execution of each phase. The project follows a **phased development approach**, ensuring that every stage — from data collection to testing and deployment — is completed with measurable progress. Scheduling and time estimation are based on an iterative development cycle, incorporating both research and implementation tasks.

4.1 Project Development Phases

The project is divided into **six key phases**, each with specific deliverables and estimated timelines:

o.	Phase Name	Description	Estimated Duration	Doliverables
	Requirement Analysis & Research	Study underwater image problems, analyze existing solutions, collect datasels (UIEB. EUVP), and finalize technical requirements.	1.5 weeks	
	System Design	Create system architecture, DFDs. ER diagram, and database design. Define preprocessing and enhancement workflow.	2 weeks	
	Model Development & Training	Implement CNN/GAN architecture, preprocess datasets, and train tte model using benchmark datasets	3 weeks	
	System Integration & Backend Development	Integrate model into backend using Flask/FastAPI, Develop APIs for upload, enhancement, and retrieval	2 weeks	
	Frontend & User Interface Development	Build responsive web/mobile interface for image uploads and viewing enhanced res-ults	2 weeks	
	Testing, Evaluation & Documentation	Conduct testing (unit, integration, and user testing), measure ppr tem performance, and prepare project documentation	1.5 weeks	

4.2 Estimated Timeline (Gantt Overview)

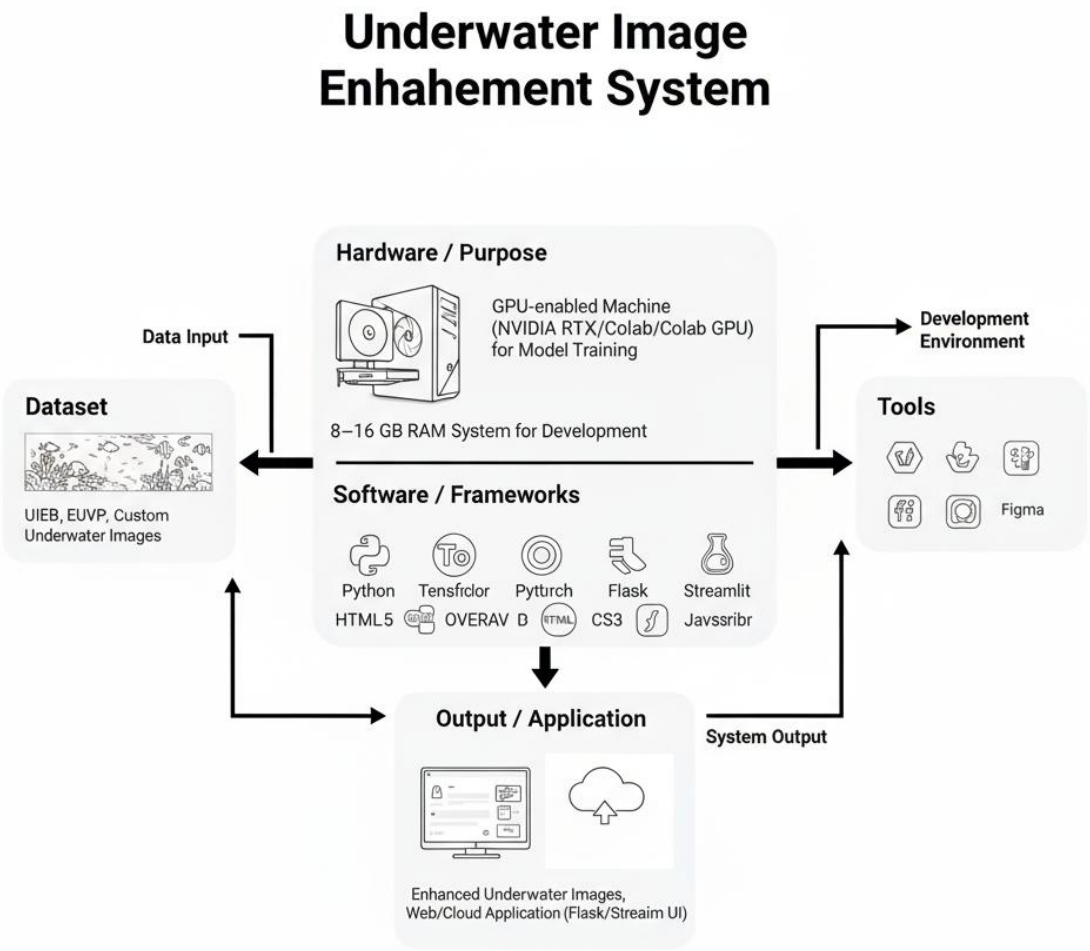


4.3 Resource Estimation

Human Resources

Role	Responsibilities
Project Lead / Devel-oper	Overall system development, model training, and deployment
	Data collection, preprocessing, and dataset management
Frontend Developer	
Tester / Evalua-tor	User interface design and web integration
	Model testing, UI testing, and performance evaluation

Hardware & Software Resources



4.4 Cost Estimation (Approximate)

Estimated Project Cost for Undrewater Image Enhahenent System

Component	Estimated Cost (NR)
Hardware Usage (GPU training hours)	₹2,000 – ₹3,000 (if using Colab Pro or AWS)
Software Tools	
Sof tware Tools	Mostly Open Source / Free
Dataset Preasting (optional)	₹0 (Public datasets)
Misuelieouious (reports, presentation, documentation)	₹1,000 – 2,000
	₹500 – 1,000
Total Estimated Cost	**₹3,000 - 6,000**

4.5 Risk & Delay Considerations

Risk Assessment and Mitigation Strategy

Risk / Impact	Mitigation Strategy
Dataset insosissienity or miages (Medium)	Use multiple benchmark datesets (UIEB, EUVP, custom samples)
GPU unalalaibaility or long training time (High)	Use cloud GPUS (Google Colab, Kaggle) for parallel training
Overfiiting or poor model performance (Medium	Implement early stopping, validation sets, and data agunertation
Deployment issues or server downtime	Use local Flask deployment before shifting to cloud
Time overruans due dubugging (Medium)	Allocate buffer time in final two weeks for testing